

Università degli Studi di Salerno

Dipartimento di Informatica
Corso di Laurea in Informatica

Tecnologie Software per il Web

Project Proposal

KeyItaly

Proposta di progetto e sistema realizzato

Docente

Prof.ssa Rita Francese

Studenti

Giovanni Cerchia	0512120818
Andrea Compagnone	0512116159

Anno Accademico 2023/2024

Indice

1	Obiettivo del progetto	1
2	Analisi dei competitor	1
2.1	Keychron	1
2.2	CoffeeKeys	2
3	Funzionalità del sito	3
3.1	Utente non registrato	3
3.2	Utente registrato	3
3.3	Amministratore	3
4	Diagramma navigazionale	4
5	Mappa dei contenuti	5
6	Base di dati	5
7	Layout	7
8	Scelta dei colori	7
9	Il sistema realizzato	9
9.1	Architettura	9
9.2	Servlet, viste e filtri	9
9.3	Dallo schema concettuale allo schema realizzato	12
9.4	Compilazione ed esecuzione	12
9.5	Limiti noti	13

1 Obiettivo del progetto

KeyItaly si propone come sito di commercio elettronico per la vendita di tastiere meccaniche e di accessori quali switch e keycaps, rivolto sia agli appassionati sia ai professionisti che cercano prodotti di qualità e personalizzabili.

Il catalogo offre tastiere adatte a esigenze diverse — dal gioco allo sviluppo software, dal lavoro d'ufficio alla scrittura — affiancate dai componenti necessari a personalizzare una tastiera già assemblata. L'interfaccia è progettata in modo responsive, così da restare utilizzabile da qualsiasi dispositivo e a qualsiasi risoluzione.

2 Analisi dei competitor

L'analisi ha riguardato due negozi di riferimento del settore, uno internazionale e uno italiano, dai quali sono state tratte le scelte di struttura e di navigazione adottate nel progetto.

2.1 Keychron

<https://www.keychron.com/>

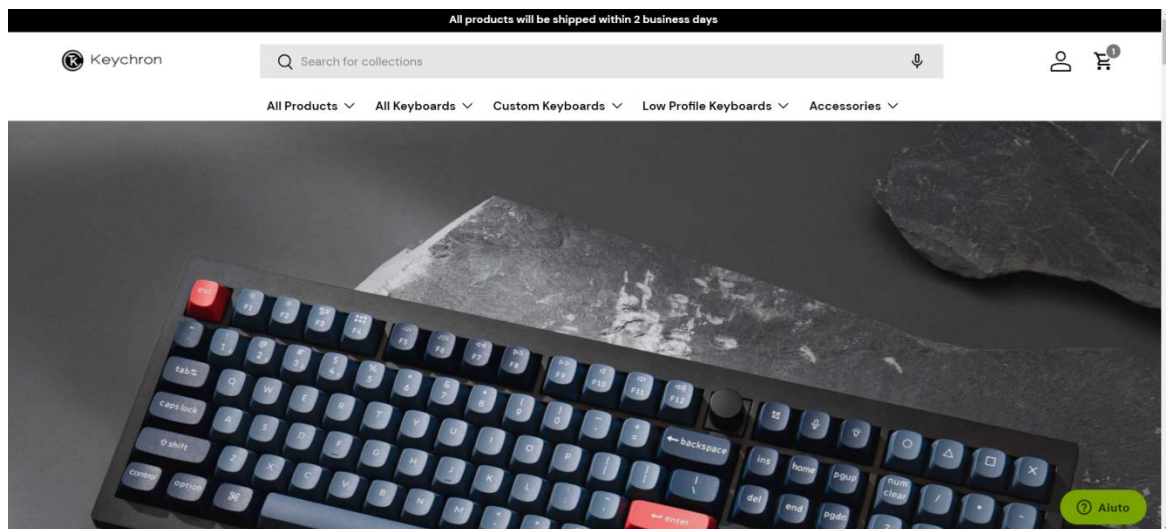


Figura 1: Pagina principale di Keychron.

Keychron è specializzato in tastiere meccaniche wireless. L'interfaccia è essenziale: un banner centrale presenta novità e prossime uscite, mentre l'header raccoglie i collegamenti al catalogo completo (*All products*), a quello delle sole tastiere (*All keyboards*), alla barra di ricerca, all'area utente e al carrello.

Il catalogo può essere filtrato per tipo, serie, layout, disponibilità e prezzo, e ordinato per nome, data, prezzo, prodotti in evidenza e più venduti. Ogni prodotto dispone di una scheda con descrizione, fotografie e selezione di colore e tipo di switch. Il carrello è utilizzabile senza autenticazione, che viene richiesta solo al momento del pagamento: è la soluzione adottata anche in *KeyItaly*.

2.2 CoffeeKeys

<https://coffeekeys.eu/>

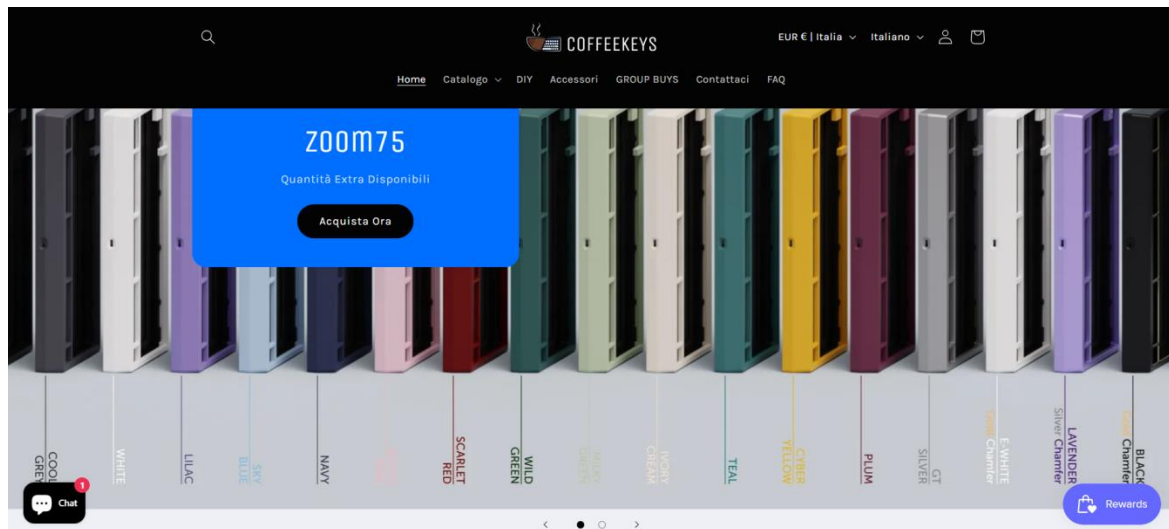


Figura 2: Pagina principale di CoffeeKeys.

CoffeeKeys è il primo negozio italiano dedicato esclusivamente alle tastiere meccaniche. La struttura è simile a quella di Keychron: banner delle novità in apertura, logo centrale affiancato da ricerca, area utente e carrello, e un menù che articola il catalogo per brand e per dimensione, con le categorie *DIY* e *Group Buys*. Completano il sito le sezioni *Accessori*, *Contattaci* e *FAQ*, e una sezione *News* con guide all'acquisto.

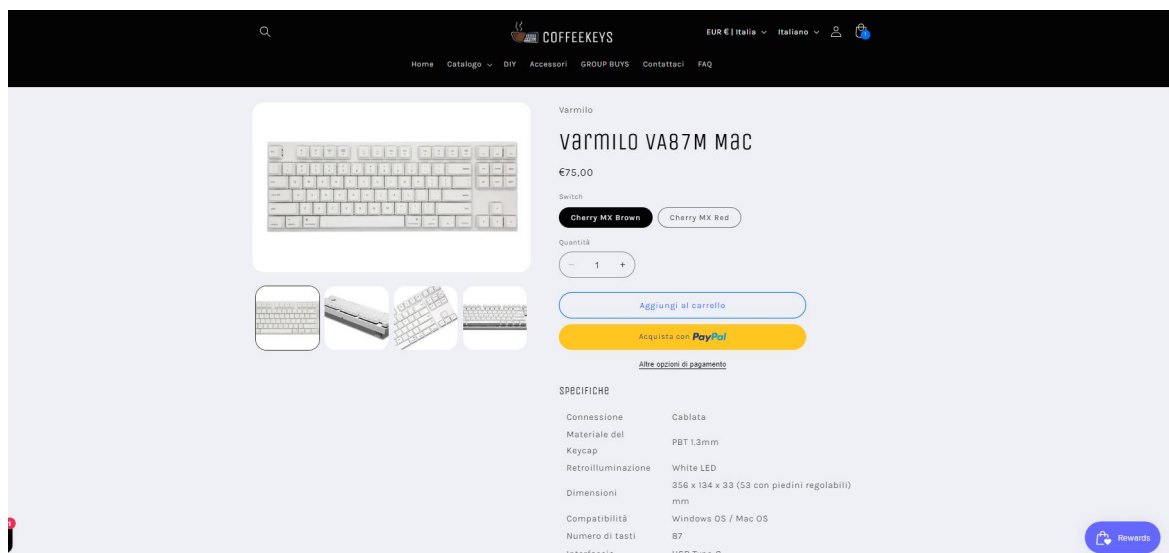


Figura 3: Scheda di prodotto su CoffeeKeys.

La scheda di prodotto raccoglie fotografie da più angolazioni, selezione di colore e switch, specifiche tecniche e articoli correlati. È il modello a cui si ispira la pagina di dettaglio di KeyItaly.

3 Funzionalità del sito

Le funzionalità sono organizzate su tre livelli di accesso.

3.1 Utente non registrato

- consultare il catalogo completo o filtrato per categoria di prodotto;
- ordinare il catalogo per nome, prezzo o tipo;
- visualizzare la pagina di dettaglio di un prodotto;
- cercare prodotti per nome;
- aggiungere e rimuovere prodotti dal carrello;
- registrarsi, operazione richiesta solo per concludere un acquisto.

3.2 Utente registrato

Oltre a quanto sopra:

- concludere un acquisto;
- consultare lo storico dei propri ordini;
- richiedere e scaricare la fattura di un ordine;
- modificare i propri dati: nome, cognome, indirizzo predefinito e metodo di pagamento.

3.3 Amministratore

- inserire, modificare ed eliminare i prodotti del catalogo tramite un'interfaccia dedicata, senza richiedere competenze di programmazione;
- consultare tutti gli ordini, filtrandoli per cliente e per intervallo di date.

4 Diagramma navigazionale

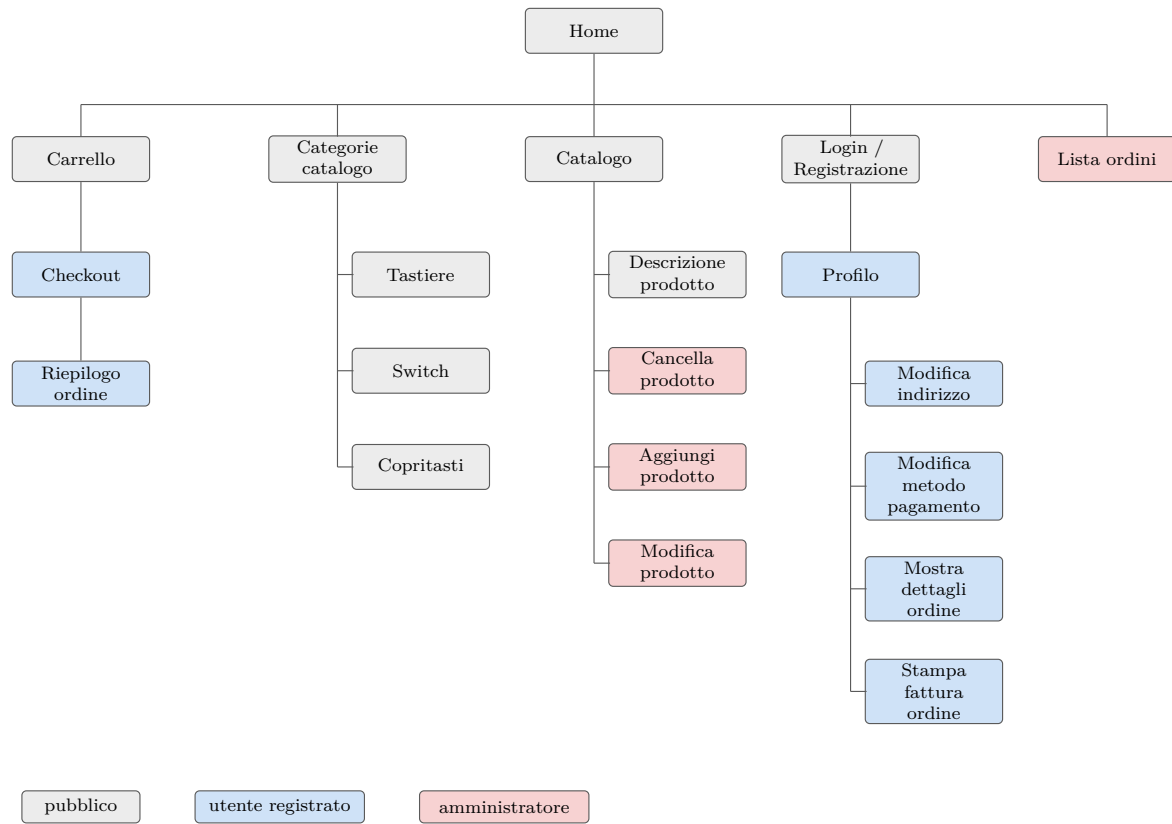


Figura 4: Diagramma navigazionale del sito.

5 Mappa dei contenuti

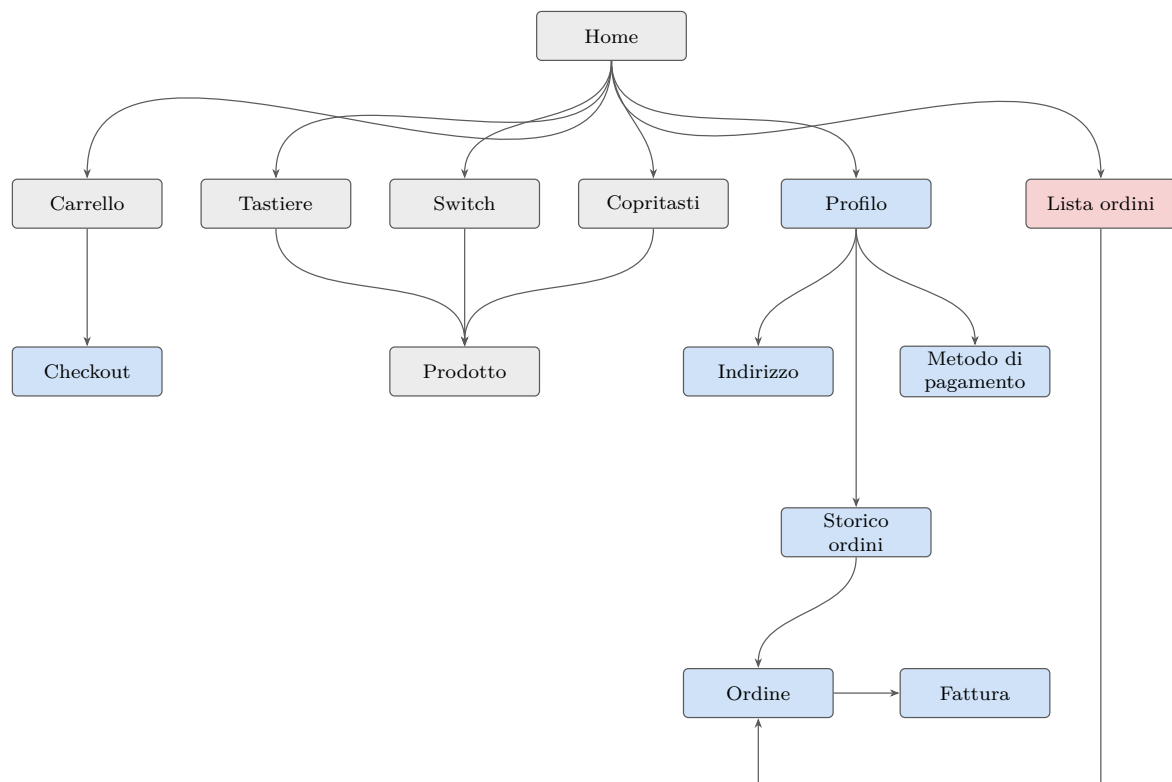


Figura 5: Mappa dei contenuti: i collegamenti percorribili fra le pagine. La codifica cromatica è quella della figura 4.

6 Base di dati

Lo schema concettuale è organizzato attorno a quattro concetti: l'utente, che si specializza in cliente e amministratore; l'ordine, eseguito da un cliente; il prodotto, a sua volta specializzato nelle tre categorie in vendita; e l'acquisto, che associa un ordine ai prodotti che lo compongono registrandone quantità, prezzo e aliquota IVA al momento dell'acquisto.

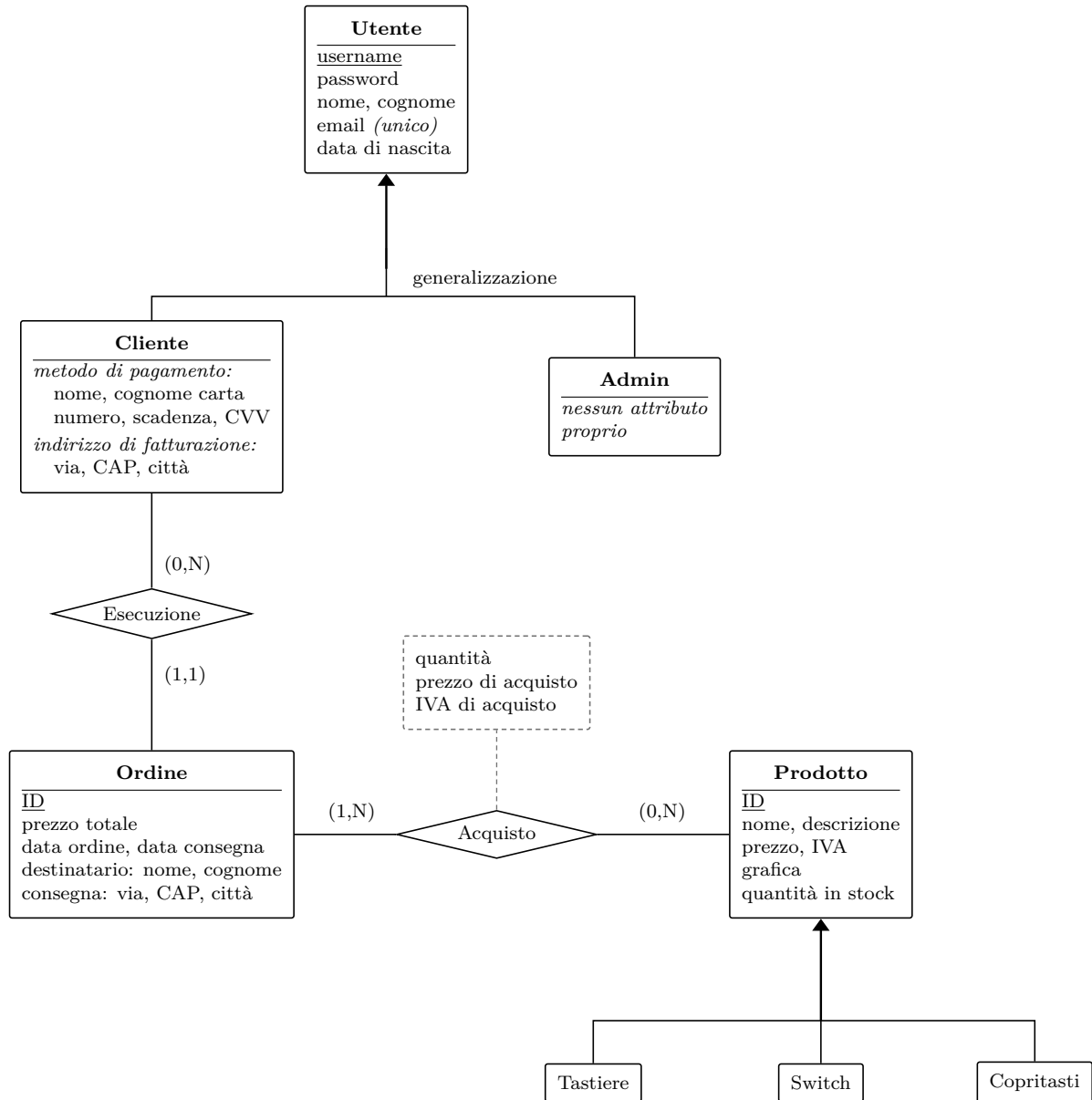


Figura 6: Schema concettuale della base di dati.

7 Layout

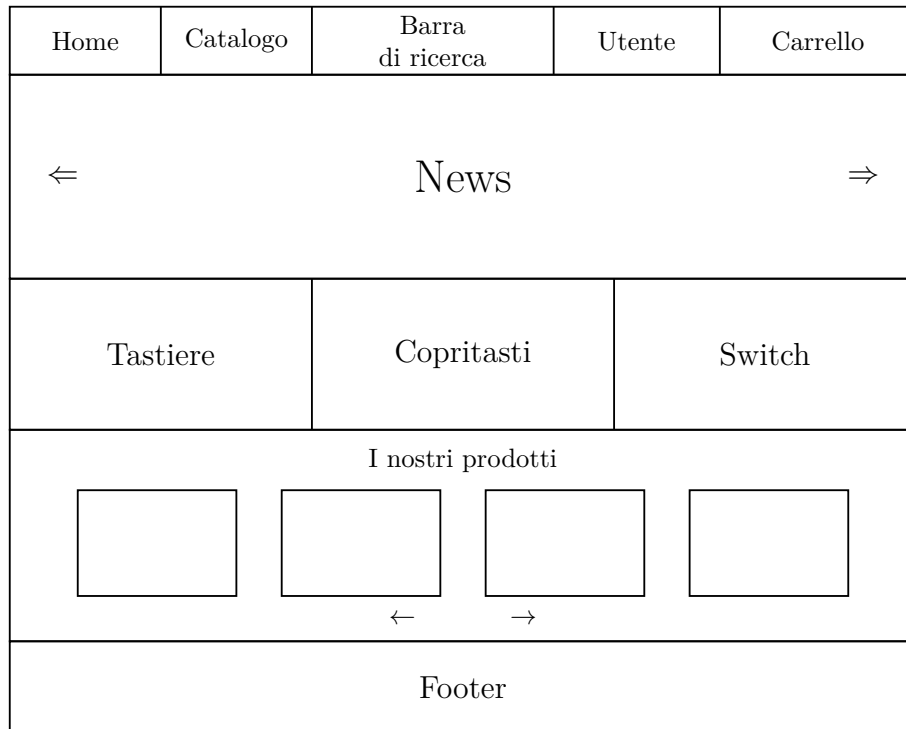


Figura 7: Wireframe della pagina principale.

8 Scelta dei colori

La palette è costruita attorno a un blu primario, impiegato per gli elementi interattivi e per gli accenti, affiancato da una scala di neutri per gli sfondi e per il testo.

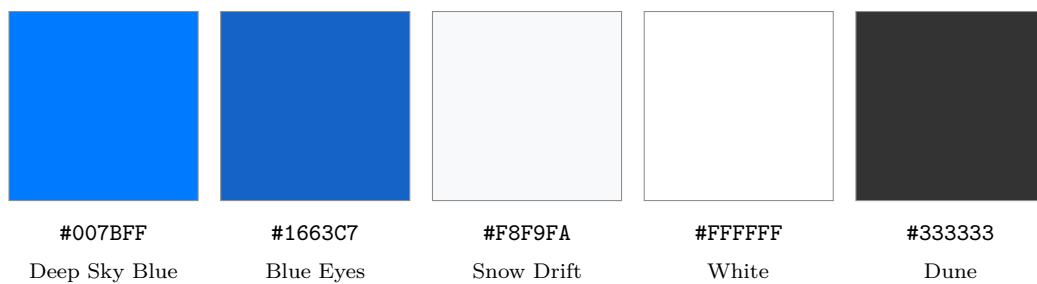


Figura 8: Palette cromatica del sito.

Colore	Codice	Impiego
Deep Sky Blue	#007BFF	Colore primario: titoli, collegamenti ed elementi in evidenza
Blue Eyes	#1663C7	Variante scura del primario, per gli stati attivi
Snow Drift	#F8F9FA	Sfondo delle pagine
White	#FFFFFF	Sfondo delle schede prodotto e dei riquadri
Dune	#333333	Testo, barra di navigazione e footer

Tabella 1: Impiego dei colori nell'interfaccia.

9 Il sistema realizzato

Questa sezione descrive sinteticamente l'applicazione effettivamente sviluppata e segnala i punti in cui si discosta dalla proposta.

9.1 Architettura

L'applicazione è scritta in Java secondo il modello *Model-View-Controller*: le servlet fanno da controller, le pagine JSP da viste e la persistenza è affidata al pattern DAO su MySQL tramite JDBC. Il pool di connessioni è dichiarato come risorsa JNDI e gestito dal contenitore, non dall'applicazione.

Componente	Versione	Ruolo
Java SE	11	Livello linguistico di compilazione
Servlet API	4.0 (<code>javax.servlet</code>)	Servlet, filtri e listener
Apache Tomcat	9.0	Contenitore web e pool di connessioni
MySQL	8.0	Base di dati
Apache PDFBox	2.0.29	Generazione delle fatture
Google Gson	2.11.0	Serializzazione JSON per gli endpoint asincroni
Apache Maven	3.9	Compilazione e produzione dell'archivio WAR

Tabella 2: Tecnologie impiegate.

Tomcat 9 è un vincolo e non una preferenza: il codice usa il namespace `javax.servlet`, che dalla versione 10 è stato rinominato in `jakarta.servlet`.

Il codice è organizzato in quattro package: `control` (21 servlet), `model` (bean e DAO), `filter` (controllo degli accessi e codifica dei caratteri) e `config` (configurazione risolta all'avvio). Nessuna pagina JSP accede direttamente alla base di dati: riceve dalla servlet i dati già pronti come attributi della richiesta o della sessione.

9.2 Servlet, viste e filtri

Il controllo degli accessi non è ripetuto in ogni servlet ma concentrato in tre filtri, a cui se ne aggiunge un quarto per la codifica dei caratteri.

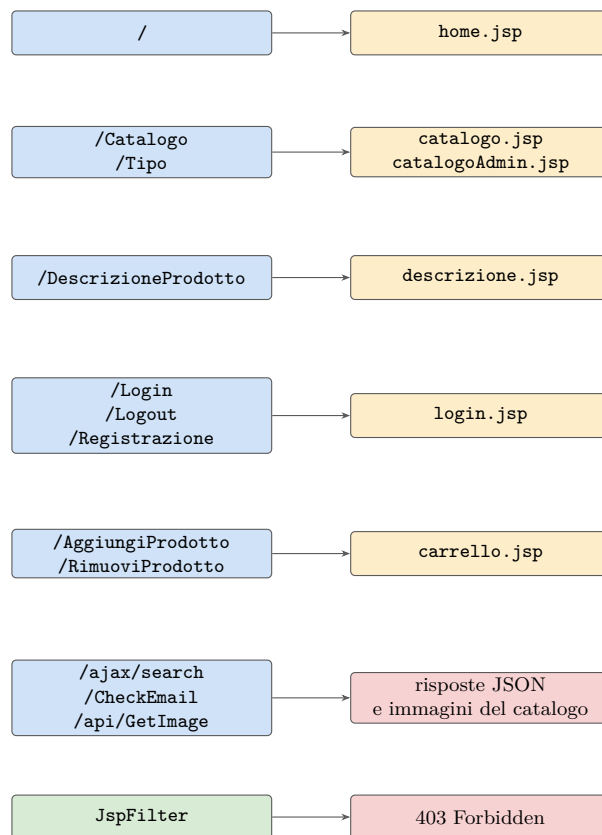
Filtro	Pattern	Comportamento
<code>EncodingFilter</code>	<code>/*</code>	Impone UTF-8 sul corpo delle richieste
<code>UserFilter</code>	<code>/user/*</code>	Richiede un utente autenticato
<code>AdminFilter</code>	<code>/admin/*</code>	Richiede il ruolo di amministratore
<code>JspFilter</code>	<code>/pages/views/*</code>	Nega ogni accesso diretto alle viste

Tabella 3: Filtri e pattern di URL.

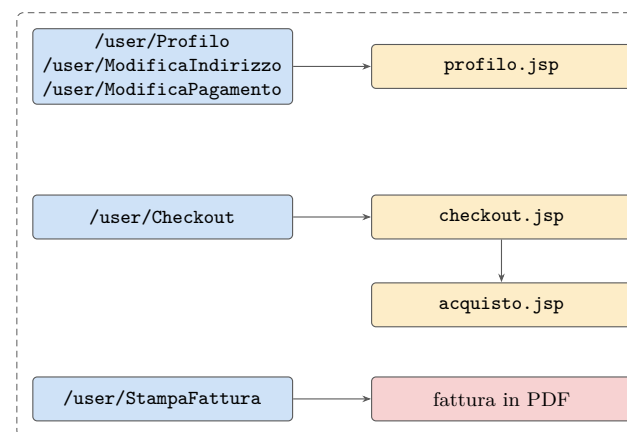
`JspFilter` merita una nota: le pagine sotto `pages/views` sono viste e non risorse pubbliche, e senza i dati preparati da una servlet produrrebbero errori. Il filtro nega quindi ogni

richiesta diretta, ma le pagine restano raggiungibili tramite l'inoltro interno, che il dispatcher predefinito non intercetta.

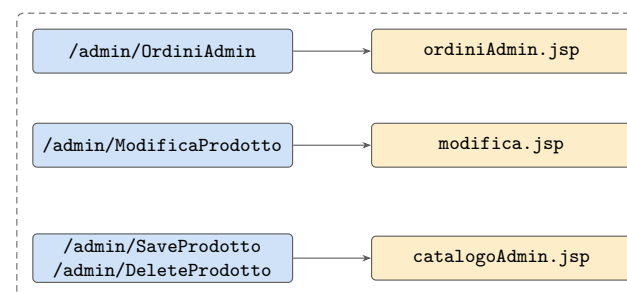
Area pubblica



ogni accesso diretto a `/pages/views/*`

Riservata a `/user/*`

protetta da `UserFilter`

Riservata a `/admin/*`

protetta da `AdminFilter`

Figura 9: Mappa delle servlet, delle viste e delle aree protette dai filtri. In azzurro le servlet, in giallo le viste JSP, in rosso le risposte generate dinamicamente, in verde il filtro sulle viste.

9.3 Dallo schema concettuale allo schema realizzato

Lo schema effettivo è più semplice di quello di figura 6, per due scelte prese in fase di sviluppo:

- **Cliente** e **Admin** non sono tabelle distinte: la specializzazione è resa da un campo **tipo** nella tabella **Utente**, che assume i valori **user** e **admin**;
- le specializzazioni di **Prodotto** non hanno attributi propri, per cui sono anch'esse ridotte a un campo **tipo** (**keyboard**, **switch**, **keycap**).

Restano quindi quattro tabelle collegate in cascata: un **Utente** ha più **Ordine**, ogni **ordine** ha più righe di **Acquisto**, e ogni riga riferisce un **Prodotto**.

Tre scelte progettuali sono degne di nota:

Dati fotografati all'acquisto. **Acquisto** non si limita a collegare ordine e prodotto: ne copia prezzo, aliquota IVA e nome del file immagine. Una variazione di listino non altera così gli ordini già conclusi, e una fattura ristampata a distanza di mesi riporta gli importi corretti.

Indirizzo per ordine. L'indirizzo di consegna è memorizzato su ogni **Ordine** e non letto dal profilo, perché lo stesso cliente può spedire a destinazioni diverse. Quello del profilo resta il valore proposto per impostazione predefinita.

Cancellazione logica. L'eliminazione di un prodotto imposta **tipo** = 'Eliminato' anziché rimuovere la riga: una cancellazione fisica violerebbe il vincolo di chiave esterna con **Acquisto** e cancellerebbe lo storico degli ordini.

9.4 Compilazione ed esecuzione

Il progetto nasceva come *Dynamic Web Project* di Eclipse: la configurazione del server, del percorso delle immagini e delle credenziali viveva nel workspace dell'IDE e non nel repository, rendendo il progetto non riproducibile altrove.

La compilazione è ora affidata a Maven, che produce l'archivio WAR e risolve le librerie in luogo dei file JAR precedentemente versionati. L'esecuzione è affidata a Docker Compose, che avvia due contenitori: MySQL, che alla prima creazione del volume esegue lo schema e i dati di esempio, e Tomcat 9 con l'applicazione, costruita da un **Dockerfile** a due fasi. Un solo comando avvia l'intero sistema, senza installare Maven, MySQL o Tomcat.

Le credenziali non sono più scritte nel repository: **context.xml** contiene segnaposto che Tomcat risolve all'avvio leggendo le variabili d'ambiente, oppure le *system property* quando l'applicazione viene avviata da IDE.

```
<Resource name="jdbc/keyItaly"
          auth="Container"
          type="javax.sql.DataSource"
          driverClassName="com.mysql.cj.jdbc.Driver"
          url="${DB_URL}"
          username="${DB_USERNAME}"
          password="${DB_PASSWORD}"
```

```
connectionProperties="useUnicode=true;characterEncoding=UTF-8"/>
```

Listing 1: Dichiarazione del data source.

Analogamente, la directory delle immagini del catalogo — esterna all'applicazione perché scrivibile dall'amministratore — viene individuata all'avvio da un listener che la cerca fra variabile d'ambiente, system property e parametro di contesto, creandola se assente. Se non è utilizzabile l'applicazione non parte: l'errore diventa così immediatamente visibile invece di manifestarsi come una serie di immagini mancanti.

9.5 Limiti noti

Il progetto ha finalità didattiche e alcune scelte, accettabili in quel contesto, non sarebbero adeguate a un sistema in esercizio.

- **Password in chiaro**, senza funzione di hash.
- **Assenza di protezione CSRF**; alcune operazioni di modifica accettano anche richieste GET.
- **Output non filtrato**: le JSP scrivono i valori senza codifica delle entità HTML, con possibile cross-site scripting persistente.
- **Giacenza non verificata**: l'acquisto non controlla né decrementa la disponibilità.
- **Nessun test automatico**: la verifica è stata condotta manualmente.

Le evoluzioni naturali sono l'introduzione dell'hashing delle password e dei token CSRF, la sostituzione degli scriptlet con JSTL ed *expression language* — che darebbe la codifica automatica dell'output — e la gestione transazionale della conclusione dell'ordine.